



PyQuest

Information- & Aufgabenblatt

Kapitel 9: Listen

Speichere mehrere Werte in einer einzigen Variable.

Listen erstellen und lesen

Eine **Liste** speichert mehrere Werte in einer einzigen Variable, in eckigen Klammern [], getrennt durch Kommas.

Auf ein Element zugreifst du über seinen **Index** – und der beginnt bei **0**, nicht bei 1!

Beispiel

```
farben = ["rot", "gruen", "blau"]  
print(farben[0])  
print(farben[2])
```

farben[0] ist das **erste** Element ("rot"), farben[2] das **dritte** ("blau"). Mit farben[-1] bekommst du das **letzte** Element, egal wie lang die Liste ist. Mit len(farben) erfährst du, wie viele Elemente die Liste hat.

Aufgabe 1: Welchen Index hat das erste Element einer Liste?

- ☐ a) 1
- ☐ b) 0
- ☐ c) -1

Aufgabe 2: Lege eine Liste **tiere** mit den Werten "Hund", "Katze", "Maus" an und gib das zweite Element aus (soll Katze sein).

Dein Code:



Listen verändern

Listen kannst du nachträglich verändern:

- `.append(wert)` fügt ein neues Element **am Ende** hinzu
- `.remove(wert)` entfernt das erste Vorkommen eines Werts
- `liste[i] = neuer_wert` ersetzt das Element an Position `i`

Ein Element hinzufügen:

Beispiel

```
einkaufsliste = ["Brot", "Milch"]  
einkaufsliste.append("Eier")  
print(einkaufsliste)
```

Beim Ausgeben einer ganzen Liste zeigt Python sie in eckigen Klammern mit allen Elementen an – genau wie du sie angelegt hast.

Aufgabe 3: Was macht `.append(wert)` bei einer Liste?

- ☐ a) Fügt den Wert am Ende der Liste hinzu
- ☐ b) Ersetzt das erste Element
- ☐ c) Löscht die Liste

Aufgabe 4: Lege eine leere Liste namen an. Füge nacheinander mit `.append()` die Werte "Ada", "Alan" und "Grace" hinzu. Gib die Liste am Ende aus.

Dein Code:



Listen mit while durchlaufen

Oft willst du **jedes Element** einer Liste bearbeiten. Mit den Bausteinen, die du schon kennst, geht das so: Du nimmst eine while-Schleife und eine **Index-Variable** *i*, die bei 0 startet.

Mit **liste[i]** holst du das Element an Position *i*, und mit **len(liste)** weißt du, wie viele Elemente es gibt. Die Schleife läuft, **solange** *i* kleiner als die Länge der Liste ist.

Jede Farbe der Reihe nach ausgeben – *i* läuft von 0 bis zum letzten gültigen Index:

Beispiel

```
farben = ["rot", "gruen", "blau"]
i = 0
while i < len(farben):
    print(farben[i])
    i += 1
```

Wichtig ist wieder das **Hochzählen** mit *i* += 1 – sonst bleibt *i* bei 0 stehen und du hast eine Endlosschleife. Die Bedingung *i* < len(farben) sorgt dafür, dass die Schleife **genau so oft** läuft, wie die Liste Elemente hat (der letzte gültige Index ist immer len(...) - 1).

Aufgabe 5: Welche Bedingung sorgt dafür, dass die while-Schleife genau alle Elemente der Liste farben durchläuft?

- ☐ a) while i < len(farben):
- ☐ b) while i <= len(farben):
- ☐ c) while farben > 0:

Aufgabe 6: Lege die Liste zahlen = [3, 7, 2, 9, 4] an. Durchlaufe sie mit einer while-Schleife und einer Index-Variable und gib nur die Zahlen aus, die größer als 4 sind.

Erwartet: 7, dann 9 (untereinander).

Dein Code:

Name: _____

Fach: IT



Das funktioniert – ist aber ganz schön umständlich mit `i`, `len()` und Hochzählen. **Merke dir das Beispiel:** Im nächsten Kapitel lernst du die **for-Schleife** kennen, mit der genau dasselbe in **einer** Zeile ohne Index gelingt. 😊